

Name: Rohit Anil Patil

Roll No: 89

Date:

Practical 7: Implement Date and Time Functions.

```
SELECT CURRENT_DATE;  
SELECT CURRENT_TIME;  
SELECT CURRENT_TIMESTAMP;  
SELECT GETDATE();  
SELECT TIME('2026-01-30 10:30:45');  
SELECT EXTRACT(YEAR FROM CURRENT_DATE);  
SELECT EXTRACT(MONTH FROM CURRENT_DATE);  
SELECT EXTRACT(DAY FROM CURRENT_DATE);  
SELECT YEAR(CURRENT_DATE);  
SELECT MONTH(CURRENT_DATE);  
SELECT DAY(CURRENT_DATE);  
SELECT DATE_ADD(CURRENT_DATE, INTERVAL 5 DAY);  
SELECT DATE_SUB(CURRENT_DATE, INTERVAL 2 MONTH);  
SELECT DATEDIFF(DAY, '2026-01-30', '2026-02-10');  
SELECT CAST('2026-01-30' AS DATE);  
SELECT MONTHNAME(CURRENT_DATE);  
SELECT WEEK(CURRENT_DATE);
```

Output:

	CURRENT_DATE
▶	2026-04-02

	CURRENT_TIME
▶	10:55:28

	CURRENT_TIMESTAMP
▶	2026-04-02 10:55:52

	GETDATE
▶	2026-04-02 10:59:39

	TIME('2026-01-30 10:30:45')
▶	10:30:45

	EXTRACT(YEAR FROM CURRENT_DATE)
▶	2026

	EXTRACT(MONTH FROM CURRENT_DATE)
▶	4

	EXTRACT(DAY FROM CURRENT_DATE)
▶	2

	YEAR(CURRENT_DATE)
▶	2026

	MONTH(CURRENT_DATE)
▶	4

	DAY(CURRENT_DATE)
▶	2

	DATE_ADD(CURRENT_DATE, INTERVAL 5 DAY)
▶	2026-04-07

	DATE_SUB(CURRENT_DATE, INTERVAL 2 MONTH)
▶	2026-02-02

	CAST('2026-01-30' AS DATE)
▶	2026-01-30

	MONTHNAME(CURRENT_DATE)
▶	April

	WEEK(CURRENT_DATE)
▶	13

Name: Rohit Anil Patil

Roll No: 89

Date:

Practical 9: Implement Nested Queries & all types of JOIN operation.

```
CREATE TABLE Departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

```
CREATE TABLE Employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    salary INT,  
    dept_id INT,  
    manager_id INT  
);
```

```
INSERT INTO Departments VALUES  
(1, 'HR'),  
(2, 'IT'),  
(3, 'Finance'),  
(4, 'Marketing');
```

```
INSERT INTO Employees VALUES  
(101, 'Amit', 50000, 2, NULL),  
(102, 'Riya', 60000, 2, 101),  
(103, 'John', 45000, 1, NULL),  
(104, 'Sara', 70000, 3, 101),
```

```
(105, 'David', 40000, NULL, NULL);
```

```
SELECT e.emp_id, e.name, d.dept_name  
FROM Employees e  
INNER JOIN Departments d  
ON e.dept_id = d.dept_id;
```

```
SELECT e.emp_id, e.name, d.dept_name  
FROM Employees e  
LEFT JOIN Departments d  
ON e.dept_id = d.dept_id;
```

```
SELECT e.emp_id, e.name, d.dept_name  
FROM Employees e  
RIGHT JOIN Departments d  
ON e.dept_id = d.dept_id;
```

```
SELECT e.emp_id, e.name, d.dept_name  
FROM Employees e  
LEFT JOIN Departments d ON e.dept_id = d.dept_id  
UNION  
SELECT e.emp_id, e.name, d.dept_name  
FROM Employees e  
RIGHT JOIN Departments d ON e.dept_id = d.dept_id;
```

```
SELECT e.name, d.dept_name  
FROM Employees e  
CROSS JOIN Departments d;  
SELECT A.name AS Employee, B.name AS Manager
```

```

FROM Employees A
LEFT JOIN Employees B
ON A.manager_id = B.emp_id;

```

Output:

- Inner Join

	emp_id	name	dept_name
▶	101	Amit	IT
	102	Riya	IT
	103	John	HR
	104	Sara	Finance

- Left Join

	emp_id	name	dept_name
▶	101	Amit	IT
	102	Riya	IT
	103	John	HR
	104	Sara	Finance
	105	David	NULL

- Right Join

	emp_id	name	dept_name
▶	103	John	HR
	102	Riya	IT
	101	Amit	IT
	104	Sara	Finance
	NULL	NULL	Marketing

- Full Join

	emp_id	name	dept_name
▶	101	Amit	IT
	102	Riya	IT
	103	John	HR
	104	Sara	Finance
	105	David	NULL
	NULL	NULL	Marketing

- Cross Join

	name	dept_name
▶	Amit	Marketing
	Amit	Finance
	Amit	IT
	Amit	HR
	Riya	Marketing
	Riya	Finance
	Riya	IT
	Riya	HR
	John	Marketing
	John	Finance
	John	IT
	John	HR
	Sara	Marketing
	Sara	Finance
	Sara	IT
	Sara	HR
	David	Marketing
	David	Finance
	David	IT
	David	HR

- Self Join

	Employee	Manager
▶	Amit	NULL
	Riya	Amit
	John	NULL
	Sara	Amit
	David	NULL

Name: Rohit Anil Patil

Roll No: 89

Date:

Practical 10: Implement practical performing different operations on a view.

```
CREATE TABLE Employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    department VARCHAR(50),  
    salary INT  
);  
  
INSERT INTO Employees VALUES  
(1, 'Amit', 'IT', 50000),  
(2, 'Neha', 'HR', 40000),  
(3, 'Raj', 'IT', 60000),  
(4, 'Simran', 'Finance', 45000);  
  
CREATE VIEW IT_View AS  
SELECT emp_id, name, salary  
FROM Employees  
WHERE department = 'IT';  
  
SELECT * FROM IT_View;  
  
INSERT INTO IT_View (emp_id, name, salary)  
VALUES (5, 'Karan', 55000);
```

```
UPDATE IT_View
SET salary = salary + 5000
WHERE emp_id = 1;
```

```
DELETE FROM IT_View
WHERE emp_id = 3;
```

```
SELECT * FROM Employees;
```

```
DROP VIEW IT_View;
```

Output:

	emp_id	name	salary
▶	1	Amit	50000
	3	Raj	60000

	emp_id	name	salary
▶	1	Amit	55000
	3	Raj	60000

	emp_id	name	department	salary
▶	1	Amit	IT	55000
	2	Neha	HR	40000
	4	Simran	Finance	45000
	5	Karan	NULL	55000
•	NULL	NULL	NULL	NULL

Name: Rohit Anil Patil

Roll No: 89

Date:

Practical 11: Implement use of Procedure.

```
CREATE TABLE Employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    department VARCHAR(50),  
    salary INT  
);
```

```
INSERT INTO Employees VALUES  
(1, 'Amit', 'IT', 50000),  
(2, 'Neha', 'HR', 40000),  
(3, 'Raj', 'IT', 60000);
```

```
DELIMITER //
```

```
CREATE PROCEDURE GetAllEmployees()  
BEGIN  
    SELECT * FROM Employees;  
END //
```

```
CREATE PROCEDURE GetEmployeeByDept(IN dept_name VARCHAR(50))  
BEGIN  
    SELECT * FROM Employees WHERE department = dept_name;  
END //
```

```
CREATE PROCEDURE UpdateSalary(IN emp INT, IN new_salary INT)
BEGIN
    UPDATE Employees SET salary = new_salary WHERE emp_id = emp;
END //
```

```
CREATE PROCEDURE DeleteEmployee(IN emp INT)
BEGIN
    DELETE FROM Employees WHERE emp_id = emp;
END //
```

```
DELIMITER ;
```

```
CALL GetAllEmployees();
```

```
CALL GetEmployeeByDept('IT');
```

```
CALL UpdateSalary(1, 70000);
```

```
CALL DeleteEmployee(2);
```

```
SELECT * FROM Employees;
```

Output:

	emp_id	name	department	salary
▶	1	Amit	IT	50000
	2	Neha	HR	40000
	3	Raj	IT	60000

	emp_id	name	department	salary
▶	1	Amit	IT	50000
	3	Raj	IT	60000

	emp_id	name	department	salary
▶	1	Amit	IT	70000
	3	Raj	IT	60000

	emp_id	name	department	salary
▶	1	Amit	IT	70000

	emp_id	name	department	salary
▶	1	Amit	IT	70000
*	NULL	NULL	NULL	NULL